

Custom Domains — Qabila

Overview

This document explains how to allow families to use a custom domain (e.g. family.example.com) to serve their family tree and pages instead of `/:tenantSlug` paths.

Key points

- Owner flow: tenant owner (role `QABILA_ADMIN`) or `SUPER_ADMIN` can set `customDomain` in the Dashboard and start verification.
- Verification: backend supports two methods:
 - DNS TXT: add a TXT record for the domain with the provided token.
 - HTTP file: create a file at `https://<customDomain>/.well-known/qabila-domain-verification.txt` containing the token.
- Only after successful verification `domainVerified` is set to `true` and incoming requests with Host equal to the custom domain will be resolved to the tenant.
- Security: sub-admins (`SUB_ADMIN`) cannot set or verify custom domains — server enforces permissions.

DNS requirements

- Preferred: create an A record pointing the custom domain to your app server IP.
- If using a CDN / proxy (Cloudflare), ensure the hostname is in DNS-only (orange cloud off) so verification via HTTP and direct TLS provisioning works (unless you let the CDN manage certificates).
- TXT verification example:
 - Name: `@` or the hostname (some DNS providers require the full host)
 - Value:

HTTP verification

- Place a plain text file at: `/.well-known/qabila-domain-verification.txt` containing only the token string.
- The backend will fetch `https://<customDomain>/.well-known/qabila-domain-verification.txt` (HTTPS recommended).
- Make sure the server serves that file without redirecting to a different host.

Reverse proxy / Host header routing

Your front-facing proxy must forward the original `Host` header to the backend (this is default for Nginx/Traefik when proxying). The API server resolves tenants using these rules (in `tenantByHost` middleware):

1. Exact match by `customDomain` (only when `domainVerified === true`).
2. If `APP_BASE_DOMAIN` is configured (e.g. `qabeelah.app`), allow subdomain matching:
`slug.APP_BASE_DOMAIN` resolves to the tenant with `subdomain === slug`.

Nginx example (proxy + Let's Encrypt)

Save file as `deployment/nginx/qabila.conf` and adapt paths/IPs.

```
server { listen 80; server_name example.com family.example.com;
```

Redirect HTTP to HTTPS

```
location /.well-known/acme-challenge/ { root /var/www/certbot; } location / { return 301  
https://$host$request_uri; } }
```

```
server { listen 443 ssl; server_name family.example.com;
```

```
ssl_certificate /etc/letsencrypt/live/family.example.com/fullchain.pem; ssl_certificate_key  
/etc/letsencrypt/live/family.example.com/privkey.pem;
```

```
location / { proxy_pass http://127.0.0.1:4000; # backend proxy_set_header Host $host; proxy_set_header X-  
Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto $scheme; }
```

Serve verification file from upstream if you want static approach

```
location /.well-known/qabila-domain-verification.txt { alias /var/www/qabila/.well-known/qabila-domain-  
verification.txt; } }
```

Notes:

- Use Certbot (or ACME client) to provision certificates for each custom domain. When many custom domains exist, consider automated tooling (see Traefik example).
- Keep `proxy_set_header Host $host;` so backend can resolve tenant by host.

Traefik (recommended for many domains)

If you run in Docker or Kubernetes, Traefik's built-in ACME and dynamic router generation is convenient. Example Docker label snippet for an app container:

docker-compose service example (short)

labels:

-

```
"traefik.http.routers.qabila.rule=Host(family.example  
.com)"
```

```
- "traefik.http.routers.qabila.entrypoints=websecure"
```

-

```
"traefik.http.routers.qabila.tls.certresolver=myresolver"
```

-

```
"traefik.http.services.qabila.loadbalancer.server.port=4000"
```

Important:

- Use TLS certificate resolver (ACME) in Traefik for automatic certs.
- You must create routers per-custom-domain (or use wildcard/HostRegexp rules).

Verification UX and lifetime

- Verification tokens are one-time and short-lived. After verification the token is cleared and `domainVerified` set to true.
- If a tenant changes domain, verification must be re-run.

Troubleshooting

- DNS TXT not found: wait for DNS propagation (can take minutes to hours). Use `dig TXT <domain>`.
- HTTP fetch failed: check your server can serve the `.well-known` path and that HTTPS works.
- Cloudflare: put the domain in DNS-only mode while verifying, or use Cloudflare's SSL and ensure origin server accepts proxied requests.

Security considerations

- Only tenant owners and platform admins can update the `customDomain` and trigger verification.
- Do not accept arbitrary redirects during verification.

API endpoints

- `POST /api/tenants/:id/domain/verify/start` — returns `{ token, instructions }`.
- `POST /api/tenants/:id/domain/verify/confirm` — body `{ method: 'dns'|'http' }`.

Example flow

1. Tenant owner sets `customDomain` in Dashboard and clicks Save.
2. Owner clicks "Start verification" — client calls `startDomainVerification` and displays instructions.
3. Owner adds TXT or HTTP file as instructed.
4. Owner clicks "Verify via DNS" or "Verify via HTTP" — client calls `confirmDomainVerification`. If successful, domain becomes active.

If you want, I can add a small admin page that lists all tenants with `customDomain` values and `domainVerified` status, and a button to re-run verification.